Now Playing Data APIs

The most important and frequently accessed pieces of information that AzuraCast stores are all served as part of a single group of data, which we refer to as the "Now Playing" data.

This data includes:

- Basic information about the station, its mount points and remote relays;
- Current listener counts, including total and unique listeners;
- The currently playing song, including a timestamp of when it was started, and whether it came from a playlist, request, or live source;
- Whether a live DJ is currently connected, and if so, what their name is; and
- A truncated history of the most recent ~5 songs (this number is customizable) played on the station.

Here's a real-world example of the Now Playing API's return data from the AzuraCast demo page.

AzuraCast always serves this data in the exact same format, regardless of whether you're broadcasting locally or remotely, or whether you're using Icecast or Shoutcast.

Because of how valuable this information is, we serve it in a number of ways depending on whether performance or flexibility is your main concern.

Standard Now Playing API

This is the "Now Playing" API endpoint listed as part of the API documentation. On any installation, an array of now-playing data for all public stations is available at:

```
http://your-azuracast-site.example.com/api/nowplaying
```

The now-playing data for a specific station is available at:

```
http://your-azuracast-site.example.com/api/nowplaying/station_shortcode
```

...replacing `station_shortcode` with either the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio") or the numeric ID of the station (visible in the URL when managing the station in AzuraCast).

Advantages

- This method is available on all installations and will always be available.
- If you have the "Prefer Browser URL" setting turned on in your installation, any URLs included in the API response (i.e. URLs for album art or listening to the station) will be updated to reflect the browser URL that was used to make the API request.
- The "elapsed" and "remaining" values on `current_song` are automatically updated to reflect their current values (in seconds) at the moment you make the API request.

Disadvantages

- Every request sent to this endpoint calls the entire Nginx/PHP-FPM stack, resulting in a very brief (under 100ms) but possibly significant performance penalty. If you have several hundred users checking this API every few seconds, the combined load can quickly overwhelm a server.

Example Implementation

Using the jQuery JavaScript library, an example implementation might look like:

```
var nowPlayingTimeout;
var nowPlaying;

function loadNowPlaying() {
    $.ajax({
        cache: false,
        dataType: "json",
        url: 'http://your-azuracast-
site.example.com/api/nowplaying/station_shortcode',
        success: function(np) {
            // Do something with the Now Playing data.
            nowPlaying = np;

            nowPlayingTimeout = setTimeout(loadNowPlaying, 15000);
        }
    }).fail(function() {
        nowPlayingTimeout = setTimeout(loadNowPlaying, 30000);
    });
}

$(function() {
    loadNowPlaying();
});
```

Using the Axios HTTP client library, an example implementation might look like:

```
var nowPlaying;
var nowPlayingTimeout;

function loadNowPlaying() {
    axios.get('http://your-azuracast-
site.example.com/api/nowplaying/station_shortcode').then((response) => {
        // Do something with the Now Playing data.
        nowPlaying = response.data;
    }).catch((error) => {
        console.error(error);
    }).then(() => {
        clearTimeout(nowPlayingTimeout);
        nowPlayingTimeout = setTimeout(checkNowPlaying, 15000);
    });
}

loadNowPlaying();
```

If your application is written in PHP, you can use the Composer package manager to install our PHP API Client, which has full support for the Now Playing API endpoints.

Static Now Playing JSON File

AzuraCast also writes its "Now Playing" API endpoint data to a static JSON file, which contains the same exact data as the standard API endpoint, but for a single station.

The Static Now Playing JSON file for a given station is available at:

```
http://your-azuracast-
site.example.com/api/nowplaying_static/station_shortcode.json
```

...replacing `station_shortcode` with the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio").

Advantages

- Because the file is static and is only changed every few seconds, Nginx, your browser, and any other reverse proxies (i.e. CloudFlare) can cache the output for a few seconds, resulting in significantly higher performance.
- Since this file's format is the same as the standard API, no code modifications or third-party libraries are needed to switch to this format.

Disadvantages

- Any URLs in the API responses will always use the "Base URL" of your AzuraCast installation.
- The "elapsed" and "remaining" durations will only be accurate as of when the file was written, not when it was downloaded by your client. You should instead compare the user's current UNIX timestamp against the `played_at` timestamp.

Example Implementation

Implementations of this method look exactly the same as for the Standard Now Playing API (above), except with the URL updated to the static URL for the station.

Simple Text File

AzuraCast also generates a simple text file containing `Artist - Title` for each station. This can be useful if you need to fetch the current playing track for display or in automation tools.

The URL of the text file follows the format:

```
http://your-azuracast-site.example.com/api/nowplaying_static/station_shortcode.txt
```

...replacing `station_shortcode` with the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio").

High-Performance Updates

We deliver a high-performance (low-latency and low server CPU burden) Now Playing feed thanks to a realtime messaging library called Centrifugo. Using this connection method, each listener gets immediate track updates while only maintaining a single lightweight HTTP connection.

Now Playing updates are delivered as unidirectional messages via Websocket, Server-Sent Events (SSE) or Long Polling web streams.

Websockets

The URL for websocket connections is:

```
wss://your-azuracast-url/api/live/nowplaying/websocket
```

Upon connection, you should send a connection string in the form of a JSON request:

```
{ "subs": { "station:your_station_name": {} }}
```

Replacing `your_station_name` with your station's URL stub or short name.

You will start to receive a feed of empty pings (`{}`) and Now Playing updates. You can identify Now Playing updates as they will have, in their parsed JSON, `pub.data.np`.

An example JavaScript implementation is below:

```
let socket = new WebSocket("wss://your-azuracast-
url/api/live/nowplaying/websocket");

socket.onopen = function(e) {
  socket.send(JSON.stringify({
    "subs": {
      "station:azuratest_radio": {"recover": true}
    }
  }));
};

let nowplaying = {};
let currentTime = 0;

// Handle a now-playing event from a station. Update your now-playing data
accordingly.
function handleSseData(ssePayload, useTime = true) {
  const jsonData = ssePayload.data;

  if (useTime && 'current_time' in jsonData) {
    currentTime = jsonData.current_time;
  }

  nowplaying = jsonData.np;
}

socket.onmessage = function(e) {
  const jsonData = JSON.parse(e.data);

  if ('connect' in jsonData) {
    const connectData = jsonData.connect;

    if ('data' in connectData) {
      // Legacy SSE data
      connectData.data.forEach(
```

```
        (initialRow) => handleSseData(initialRow)
      );
    } else {
      // New Centrifugo time format
      if ('time' in connectData) {
        currentTime = Math.floor(connectData.time / 1000);
      }

      // New Centrifugo cached NowPlaying initial push.
      for (const subName in connectData.subs) {
        const sub = connectData.subs[subName];
        if ('publications' in sub && sub.publications.length > 0) {
          sub.publications.forEach((initialRow) => handleSseData(initialRow,
false));
        }
      }
    }
  } else if ('pub' in jsonData) {
    handleSseData(jsonData.pub);
  }
};
```

Server-Sent Events (SSE/EventSource)

The URL for SSE connections is:

```
https://your-azuracast-url/api/live/nowplaying/sse?cf_connect=TOKEN
```

Where the `cf_connect` URL parameter is the same connection token as the first message in the Websocket example.

An example JavaScript implementation is below:

```
const sseBaseUri = "https://your-azuracast-url/api/live/nowplaying/sse";
const sseUriParams = new URLSearchParams({
  "cf_connect": JSON.stringify({
    "subs": {
      "station:azuratest_radio": {"recover": true}
    }
  })
});
const sseUri = sseBaseUri+"?"+sseUriParams.toString();
const sse = new EventSource(sseUri);

let nowplaying = {};
let currentTime = 0;

// This is a now-playing event from a station. Update your now-playing data
accordingly.
function handleSseData(ssePayload, useTime = true) {
  const jsonData = ssePayload.data;

  if (useTime && 'current_time' in jsonData) {
    currentTime = jsonData.current_time;
  }

  nowplaying = jsonData.np;
}

sse.onmessage = (e) => {
  const jsonData = JSON.parse(e.data);

  if ('connect' in jsonData) {
    const connectData = jsonData.connect;

    if ('data' in connectData) {
      // Legacy SSE data
      connectData.data.forEach(
```

```
        (initialRow) => handleSseData(initialRow)
      );
    } else {
      // New Centrifugo time format
      if ('time' in connectData) {
        currentTime = Math.floor(connectData.time / 1000);
      }

      // New Centrifugo cached NowPlaying initial push.
      for (const subName in connectData.subs) {
        const sub = connectData.subs[subName];
        if ('publications' in sub && sub.publications.length > 0) {
          sub.publications.forEach((initialRow) => handleSseData(initialRow,
false));
        }
      }
    }
  } else if ('pub' in jsonData) {
    handleSseData(jsonData.pub);
  }
};
```